

Experiment 5: Latches, Flip-Flops and Counters

Digital Systems and Microcontrollers, IIIT-H

Monsoon'26

1 Introduction

In this experiment, the two basic modules required for the design of sequential circuits – LATCH for asynchronous circuits, and for synchronous circuits – FLIP-FLOP will be studied. RS latch using NOR gates will be separately assembled and tested. The basic operation of master-slave J-K flip-flop will then be studied by combining two latches to make a flip-flop. All IC's used in this experiment belong to the CMOS CD40xx family.

2 Part A: RS Latch

- Assemble a NOR latch using two gates from the CD4001 chip, with the R and S inputs connected to two Input Switches and the Q and Q' outputs displayed on two LED Displays.
- Observe and tabulate the sequence of Q and Q' in response to the following input sequence: S R = 01, 00, 10, 00, 01, 10, 01, 00, 11, 00, 10, 11, 00, 01, 11, 00.
- Interpret the observed result and verify the latch operation.

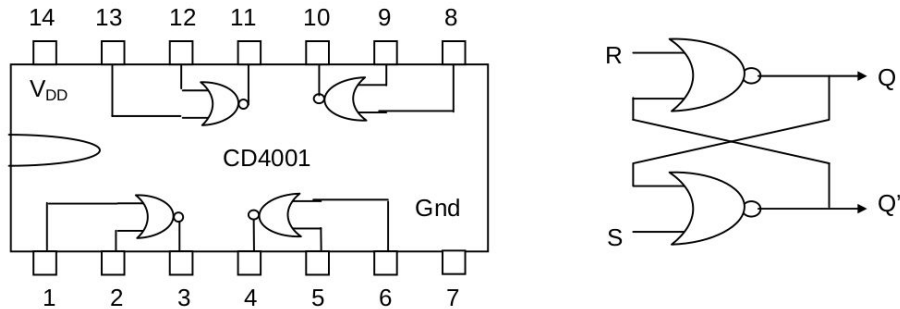


Figure 1: NOR Latch using CD4001 Quad 2-input NOR gate

3 Part B: JK Master-Slave Flip-Flop

- Assemble the circuit as shown in Figure 2 by using two 4-input NAND gates from a CD4012 chip as 3-input gates.
- Connect the unused inputs of the 4-input NAND gates to the HIGH (VCC) level.

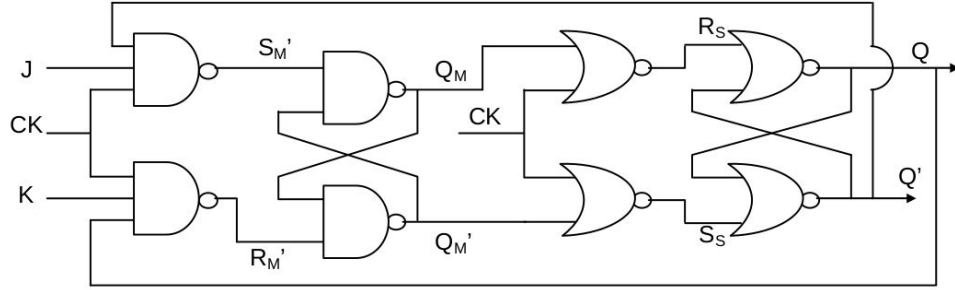


Figure 2: J-K Master-Slave Flip-Flop Circuit

- Verify theoretically that the circuit does realise the required logic for the R-S inputs of the Master and Slave latches constituting a J-K Master-Slave flip-flop: TODO

$$R_M = K \cdot Q \cdot CK, S_M = J \cdot Q' \cdot CK, S_S = Q_M \cdot CK' \text{ and } R_S = Q'_M \cdot CK'$$

J	K	ACTION	Q_{n+1}
0	0	HOLD	Q_n
0	1	CLEAR	0
1	0	SET	1
1	1	TOGGLE	Q'_n

Table 1: J-K Flip-Flop Truth Table

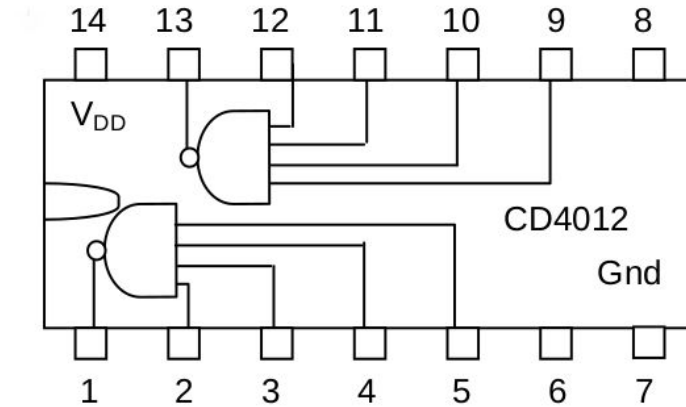


Figure 3: CD4012 Pinout

4 Part C: 4-bit Up-Down Counter

1. In the second part of the experiment, we will use an Arduino to implement a 4-bit ripple counter. The bits will be represented by LEDs.
2. Connect Arduino pins to the LEDs on the breadboard and set the pins to OUTPUT.

```

void setup()
{
    pinMode(ledPin, OUTPUT);
}

```

3. First initialize a `Timer t` and use the `t.oscillate` function in Arduino to toggle the pin value after a particular period.

`int oscillate(int pin, long period, int startingValue)`: Toggle the state of the digital output `pin` every `period` milliseconds. E.g. `t.oscillate(13,500,LOW)` will toggle output pin 13, which will toggle every 500 ms and will start from LOW (0).

Hint: Make a function which will initialize `t.oscillate` for the four pins.

4. Write a code which will behave as a 4-bit counter and change the value of the output pins accordingly which should be seen on the LEDs.
5. Down counter: Add an if condition in the code to switch the code to a down counter once the count reaches 16 (1111 in binary). Hint: when switching, first stop the timer and then initialize it again for the down counter.

end here